
Decoupling What from Where: How Should a Small GUI Grounding Model Receive the Action Type?

Aadi Chauhan*
Stanford University
aadic@stanford.edu

Arthur Ilyasov*
Stanford University
ailyasov@stanford.edu

Abstract

A GUI agent decides which action to take and where on the screen to take it; we ask how a small grounding model should receive the action type, and what the type buys it. Fine-tuning Qwen2-VL-2B with LoRA on Android in the Wild, we compare a flat baseline with five ways of supplying the type under matched data, compute, adapters, and decoding: an auxiliary loss, a hard-routed action word, an additive learned embedding, a prepended learned token, and the type written into the prompt. With five seeds, a bootstrap that clusters validation steps by episode, and seed-level paired tests, the ranking on a stream of clicks, scrolls, and type events is clear: the auxiliary loss, the additive embedding, and the prompt word each gain five to seven hit@0.10 points over the baseline, and hard routing and the prepended token are not distinguishable from it. Much of that gain is protection from a preprocessing choice of ours rather than a spatial prior. Our serializer clamps the off-screen touch point that AITW records for type events to the origin; that class degrades the flat baseline’s click grounding, and retraining without it lifts the baseline by nearly seven points, after which no mechanism’s hit rate beats it and the intervals are tight enough to exclude a two-point effect, though the auxiliary loss still shortens the average miss; on a stream of taps and swipes none helps. Whether this is a quirk of one serialization or a general hazard is open. For deployment, the pipeline’s margin over the baseline with predicted rather than gold types is not established (+0.016, 95% interval $[-0.017, +0.052]$), and a wrong type collapses every model conditioned at inference. The prepended token does not help at the shared learning rate, where its rows barely move from initialization; trained ten times faster it reaches the level of the other three, with a margin over the baseline that three seeds do not establish. We also document a silent failure: conditioning injected through `inputs_embeds` makes Qwen2-VL fall back to one-dimensional positions for its image tokens, which cost nine points.

1 Introduction

A GUI agent maps a screenshot and an instruction to a low-level action: an action type such as *click*, *scroll*, or *type*, and, for spatial actions, a target location. Native agent models (UI-TARS [Qin et al., 2025], OS-Atlas [Wu et al., 2025d], SeeClick [Cheng et al., 2024], ShowUI [Lin et al., 2025]) produce both in a single autoregressive stream. The two decisions are different kinds of prediction: the type is a small classification problem, and the location is a localization problem that has to find the evidence on the screen and commit to a point. Several agents therefore predict the type first and the target conditioned on it [Ma et al., 2024, Christianos et al., 2025, Wu et al., 2025c], and CoCo-Agent ablates its conditional prediction. What is missing is a comparison of the ways of delivering

*Equal contribution.

the type under matched data and compute, and a test of whether the trained model reads the signal at all.

This paper is that comparison, in a small model. We fine-tune Qwen2-VL-2B [Wang et al., 2024] with LoRA [Hu et al., 2022] to emit a coordinate string and compare a flat baseline with five conditioning mechanisms that see the same data in the same order under the same optimizer, adapters, and decoding. We started from the hypothesis that the prepended token would be the right mechanism, because it gives the model a dedicated, attendable position for the type.

The ranking is clear and our hypothesis was wrong. On a stream that mixes clicks, scrolls, and type events, the auxiliary loss, the additive embedding, and the prompt word raise hit@0.10 by five to seven points over the baseline (five seeds, episode-clustered intervals, seed-level $p \leq 0.03$), almost entirely on clicks; hard routing and the prepended token are within noise of it. The gain comes from a preprocessing choice of ours: AITW records type events with an off-screen touch point that our serializer clamps to the origin, so the flat model must predict a fixed point for a class it is not told about. Exact emissions of that point explain little of the gap; the class pulls the baseline’s clicks toward the origin more broadly, retraining without it raises the baseline by nearly seven points, after which no mechanism beats it on the headline metric by more than 0.002, and on a separate stream of taps and swipes none helps. With a Stage-1 classifier at 84% accuracy the pipeline’s margin over the flat baseline is not established, and a wrong type collapses every model conditioned at inference.

Interventions on the trained models separate training effects from inference effects: a wrong or blended type costs both embedding models most of their accuracy, so both read their embedding, and the zeroed additive model is no more than a point below the flat baseline whereas the zeroed prepended-token model is seven points below it. The prepended token’s learned rows barely move from their random initialization under the shared learning rate; trained ten times faster they move, and the token reaches the level of the other three mechanisms, so its failure is a statement about the schedule, not the architecture. An earlier implementation that injected the conditioning through `inputs_embeds` lost nine points to a silent positional-encoding fallback that we first read as evidence against conditioning.

Our contributions are (1) a matched-compute comparison of five ways of delivering the action type, with five seeds, paired statistics at the episode and seed levels, and a control stream; (2) a diagnosis of where the mixed-stream gain comes from, including retraining with the degenerate class removed; (3) a wrong, zero, and class-mean intervention protocol that tests whether a conditioning signal is read and what remains when it is removed; and (4) an account, checked against the library source, of a positional-encoding failure that inverted a conclusion.

2 Related Work

GUI grounding and agent models. Mind2Web [Deng et al., 2023] and Android in the Wild [Rawles et al., 2023] supply our episodes; AndroidControl [Li et al., 2024] and AndroidWorld [Rawles et al., 2025] extend them. SeeClick [Cheng et al., 2024] introduced ScreenSpot (extended by ScreenSpot-Pro [Li et al., 2025]), and UGround [Gou et al., 2025] and OS-Atlas [Wu et al., 2025d] scale grounding pretraining. Aguvis [Xu et al., 2025], UI-TARS [Qin et al., 2025], CogAgent [Hong et al., 2024], ScreenAI [Baechler et al., 2024], and Ferret-UI [You et al., 2024b] emit action and location together, the design our flat baseline follows; TinyClick [Pawlowski et al., 2024] and Ferret-UI Lite [Yang et al., 2025b] work at our scale. GUI-Actor [Wu et al., 2025a] replaces coordinate text with an attention head, MolmoPoint [Clark et al., 2026] and GUI-AIMA [Zhou et al., 2025] add grounding or anchor tokens, UI-R1 [Lu et al., 2026] rewards type and argument separately, and OmniParser [Lu et al., 2024] and Set-of-Mark [Yang et al., 2023] externalize grounding. Wang et al. [2025a] argue that the implicit mapping from rotary position to coordinate limits grounding and add explicit coordinate markers with an interleaved M-RoPE; Section 6.2 shows a cruder way the same positions go wrong.

Type-first agents. CoCo-Agent [Ma et al., 2024] predicts the type and then the target conditioned on it in one stream, our hard-routing variant. LiMAC [Christianos et al., 2025] places a small action transformer ahead of a LoRA-tuned Qwen2-VL-2B on AITW and AndroidControl; the action transformer predicts the type and selects click targets contrastively over UI-element embeddings, and the VLM generates only text arguments. AutoGLM [Liu et al., 2024] and Hi-Agent [Wu et al., 2025c]

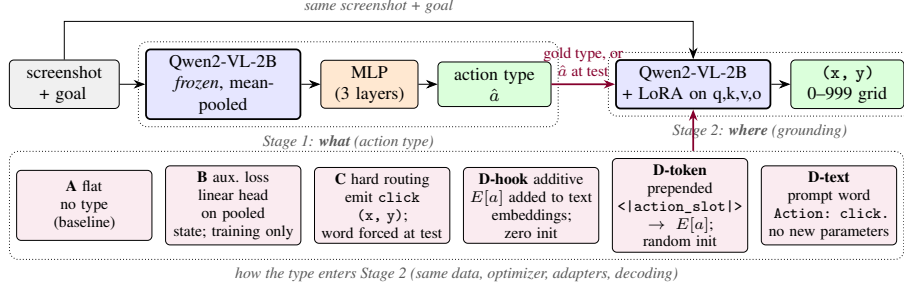


Figure 1: Two-stage pipeline. Stage 1 classifies the action type from mean-pooled frozen features. Stage 2 grounds the action to a coordinate string, and the action type enters it through one of the six mechanisms in the bottom row, all of which share data, optimizer, adapters, and decoding. D-slot, the retracted variant whose `inputs_embeds` injection bypassed the position computation, is described in Section 6.2.

separate planning from low-level action, as do SeeAct [Zheng et al., 2024], Ponder & Press [Wang et al., 2025b], and Aria-UI [Yang et al., 2025a]. None holds data, compute, and decoding fixed across ways of delivering the type, or intervenes on the trained model to ask whether the signal is used.

Conditioning, faithfulness, and the backbone. SayCan [Ahn et al., 2022], ReAct [Yao et al., 2023], and RT-2 [Brohan et al., 2023] factor, expose, or tokenize the action decision; AutoUI [Zhang and Zhang, 2024], AppAgent [Zhang et al., 2025], DigiRL [Bai et al., 2024], WebRL [Qi et al., 2025], VSC-RL [Wu et al., 2025b], and GUI-Libra [Yang et al., 2026] improve mobile agents without changing how the type enters grounding, and our auxiliary loss is multi-task learning in the sense of Caruana [1997]. Kosmos-2 [Peng et al., 2024], Shikra [Chen et al., 2023], and Ferret [You et al., 2024a] teach VLMs to refer to regions; POPE [Li et al., 2023] and HallusionBench [Guan et al., 2024] ask whether an output is supported by the image and our attention measurements ask that of a coordinate, while Lin et al. [2026] report that grounding is largely fixed during prefill. Qwen2-VL [Wang et al., 2024] assigns three-dimensional rotary positions (extending RoPE [Su et al., 2024]) to visual tokens from `input_ids`; Section 6.2 describes what happens when conditioning bypasses that path.

3 Method

3.1 Action taxonomy and grounding target

Both datasets are mapped onto a canonical eight-class action space (*click*, *double-click*, *type*, *scroll*, *drag*, *hotkey*, *wait*, *finished*). Mind2Web’s CLICK, SELECT, and HOVER map to *click* and TYPE to *type*. AITW stores gestures as touch and lift points; we split them into taps (*click*) and swipes (*scroll*) by the normalized distance between the two, with threshold 0.04. The grounding target is the touch point normalized to $[0, 1]^2$, serialized as (x, y) with integers on a 0–999 grid so every target has the same token length. AITW records non-touch actions such as typing with the touch point $(-1, -1)$; the serializer clamps this to $(0, 0)$, so every *type* event in the training stream has the same target at the screen origin. Its distance from the raw target is at least $\sqrt{2}$ for any on-screen prediction, so no model can score a hit on a type event; we keep these events in the validation slices, where every model scores zero on them, and Section 5.2 measures what they do to training.

3.2 Stage 1: which action

Stage 1 runs the frozen Qwen2-VL-2B-Instruct backbone over the screenshot and instruction, mean-pools the final hidden states over unmasked tokens into a 1536-dimensional vector, and trains a three-layer MLP ($1536 \rightarrow 1024 \rightarrow 256 \rightarrow 8$, dropout 0.1) with class-weighted cross-entropy (AdamW, learning rate 10^{-4} , weight decay 0.01, batch size 128, eight epochs, best validation macro-F1 checkpoint). Features come from text only, text plus screenshot, or a zeroed sanity variant.

3.3 Stage 2: where

Stage 2 fine-tunes the same backbone with LoRA adapters (rank 16, $\alpha = 32$, dropout 0.05) on the query, key, value, and output projections, 4.4M trainable parameters out of 2.2B. The prompt is the screenshot, `Goal: {goal}`, and a fixed instruction to predict the action coordinate; the answer is the coordinate string. With s the screenshot, g the goal, a the action type, and $y_{1:T}$ the coordinate tokens, every variant minimizes $\mathcal{L}_{\text{coord}} = -\sum_{t=1}^T \log p_{\theta}(y_t \mid y_{<t}, s, g, c)$, masked to the answer tokens, with c the variant-specific conditioning.

A, flat. $c = \emptyset$: the joint-decoding design of native agent models, reduced to the grounding sub-problem.

B, auxiliary loss. A linear head h_{ϕ} on the mean-pooled last hidden state $\bar{z}$ predicts the action type during training, $\mathcal{L}_B = \mathcal{L}_{\text{coord}} + \lambda \text{CE}(h_{\phi}(\bar{z}), a)$ with $\lambda = 1$. The mean runs over every unmasked position of the teacher-forced sequence, answer tokens included, so the head can read the type off the answer (a type event’s answer is (0, 0)). Nothing about the type is provided at inference, so any gain is a training effect.

C, hard routing. The model is trained to emit the action word before the coordinate, `click (x, y)`, as in CoCo-Agent’s conditional action prediction. At test time the word for the gold (or Stage-1) type is forced as a decoding prefix.

D-hook, additive embedding. A learned table $E \in \mathbb{R}^{8 \times d}$ holds one vector per class. A forward hook on the text-embedding module adds $E[a]$ to the embedding of every position in `input_ids`; Qwen2-VL then overwrites the image-placeholder positions with the visual features, so the bias reaches the text tokens (the prompt and, in training, the answer) and not the image tokens. E is zero-initialized, so D-hook starts as an exact copy of A.

D-token, prepended action token. A new token `<|action_slot|>` is added to the vocabulary and placed at the start of the text prompt, so it occupies a real position in `input_ids`; a forward hook replaces its embedding with $E[a]$, initialized from $\mathcal{N}(0, 0.02^2)$. This is the architecture we originally hypothesized: a full-norm, attendable representation of the type. The table shares the base learning rate; Section 5.1 also trains it ten and a hundred times faster.

D-text, type as a word. The prompt begins with `Action: click.` (or the gold type’s word), using the pretrained word embeddings and no new parameters: the obvious baseline for the two learned-embedding variants.

For C, D-hook, D-token, and D-text the gold type is used in training and at evaluation unless stated otherwise; Section 5.4 closes the loop with Stage-1 predictions. All variants share the data order, the optimizer (AdamW, learning rate 2×10^{-5} , weight decay 0.01, batch size 1, gradient clipping at 1.0, two epochs), and greedy decoding with 16 new tokens. No hyperparameter was tuned per variant.

4 Experimental setup

Data. Stage 1 uses 7,775 Multimodal-Mind2Web training steps evaluated on the 1,338-step `test_task` split, and 5,000 AITW General steps evaluated on 1,000 held-out steps. Stage 2 uses two AITW streams, built by filtering the training split of the `cjfcst/AITW_General` mirror in its stored order. `all_with_coords` keeps taps, swipes, and type events; at 1,200 training examples its validation slice of 250 steps holds 169 clicks, 46 scrolls, and 35 type events from 42 episodes. `taps_and_swipes` keeps only taps and swipes (1,000 training and 200 validation steps from 37 episodes), which land anywhere on the screen. For a given mix and size every seed and variant sees the same examples in the same order, which makes paired tests possible; the validation slice begins where the training slice ends, so absolute numbers are not comparable across sizes. This is not the official episode split and no action history is given, which limits comparability with published numbers, not the matched comparison. A second control uses Multimodal-Mind2Web (1,000 training and 250 evaluation steps; the target is the gold element’s center). Stage 2 is studied on AITW because there the type is a vision problem: Stage 1 gains +0.414 macro-F1 from the screenshot on AITW and +0.009 on Mind2Web, whose instructions name the action (Table 5).

Metrics. `hit@r` is the fraction of predictions within normalized Euclidean distance r of the target, for $r \in \{0.05, 0.10, 0.25\}$ and $r = 0.14$, the tap threshold of AITW’s matcher [Rawles et al., 2023];

Table 1: Stage 2 grounding on AITW all_with_coords ($n_{\text{train}}=1200$, $n_{\text{val}}=250$, 5 seeds). Mean $\pm$ std over seeds; Δ vs. A with a 95% episode-cluster bootstrap CI (stars from the same bootstrap, $*p < .05$, $**p < .01$, $***p < .001$) and the seed-level paired t -test p .

Variant	hit@0.05	hit@0.10	hit@0.25	mean L2 $\downarrow$	Δ hit@0.10 vs. A [episode CI]	seed-level p
A: flat	0.130 $\pm$ 0.039	0.229 $\pm$ 0.043	0.499 $\pm$ 0.029	0.406 $\pm$ 0.028		
B: aux. loss	0.178 $\pm$ 0.025	0.293 $\pm$ 0.021	0.582 $\pm$ 0.026	0.365 $\pm$ 0.004	+0.064 [+0.037, +0.092]***	0.004
C: hard routing	0.170 $\pm$ 0.022	0.264 $\pm$ 0.045	0.538 $\pm$ 0.059	0.421 $\pm$ 0.016	+0.035 [-0.002, +0.072]	0.189
D-hook: additive	0.170 $\pm$ 0.042	0.282 $\pm$ 0.041	0.563 $\pm$ 0.016	0.370 $\pm$ 0.010	+0.054 [+0.022, +0.090]***	0.028
D-token: prepended	0.144 $\pm$ 0.056	0.238 $\pm$ 0.049	0.504 $\pm$ 0.055	0.386 $\pm$ 0.021	+0.009 [-0.019, +0.036]	0.465
D-text: word in prompt	0.186 $\pm$ 0.022	0.302 $\pm$ 0.028	0.568 $\pm$ 0.033	0.364 $\pm$ 0.008	+0.073 [+0.030, +0.117]**	0.018

hit@0.10 is the headline, and we report the mean normalized distance. Unparseable outputs count as misses at distance $\sqrt{2}$; only hard routing produces them, and Section 5.1 reports its numbers on parsed outputs as well.

Statistics. The headline setting uses five seeds (42 to 46); every other cell uses three (42 to 44), and we report the mean and sample standard deviation over seeds. Every run logs a per-example distance on a shared evaluation set with a shared per-seed data order, so comparisons are paired at two levels. Steps of one episode share app, goal, and near-identical screens, so our primary interval is a cluster bootstrap over episodes, resampling episodes with replacement and keeping every step and seed of a sampled episode together (10,000 resamples, 95% percentile interval, two-sided bootstrap p [Efron and Tibshirani, 1993]). One episode straddles the training/validation boundary in every all_with_coords slice; dropping its steps moves no estimate by more than 0.003 (Appendix A). A paired t -test on the five (or three) per-seed means has little power but cannot be fooled by within-seed correlation; we call a difference established only when both agree, and stars come from the episode bootstrap with the seed-level p beside them. No correction is made for the more than fifty contrasts reported; the scaling study (Appendix D) is descriptive for that reason.

5 Results

5.1 On the mixed stream, three mechanisms beat the flat baseline and two do not

Table 1 is the main comparison: six variants on all_with_coords at 1,200 training examples, five seeds each, with episode-clustered intervals. Three mechanisms beat the flat baseline on every hit rate and on mean distance: the auxiliary loss (B) by +0.064 hit@0.10 (95% interval [+0.037, +0.092], seed-level $p = 0.004$), the additive embedding (D-hook) by +0.054 ([+0.022, +0.090], $p = 0.028$), and the type written into the prompt (D-text) by +0.073 ([+0.030, +0.117], $p = 0.018$). The three are indistinguishable (D-hook - B: -0.010, [-0.043, +0.024]; D-text - B: +0.009, [-0.040, +0.058]). Hard routing (C) gains +0.035 with an interval through zero ([-0.002, +0.072], $p = 0.19$) and loses on mean distance because some forced outputs do not parse (Table 2). The prepended token (D-token), the mechanism we set out to validate, is within noise of the baseline on every hit rate (+0.009, [-0.019, +0.036]) and below B by -0.055 ([-0.083, -0.026], $p = 0.016$). At AITW’s 0.14 tap threshold the picture is the same (B +0.059, D-hook +0.042, D-text +0.067, intervals excluding zero and seed-level $p \leq 0.04$; C +0.018 and D-token +0.003 with intervals through zero). Seed variance is large (the baseline spans 0.169 to 0.280), hence paired statistics rather than a best run. The headline cell had three seeds when the study was designed; the fourth and fifth seeds and D-text were added after the three-seed ordering was known, and it did not change.

D-token’s failure is confounded by an untrained embedding: at the shared learning rate its rows move by 0.016 from an initialization of norm 0.78 (Table 9), so Table 1 tests a fixed random class code plus LoRA. Training the table ten and a hundred times faster moves the rows by 0.28 and 1.9 and raises hit@0.10 to 0.295 and 0.283 (three seeds), within noise of B; the margin over the baseline, +0.040 ([+0.005, +0.076], seed $p = 0.16$) and +0.028 ([-0.008, +0.065], seed $p = 0.02$), passes one of our two tests each time and not the other. We therefore call the trained token level with the other conditioned models and its advantage over the baseline suggestive: the refutation is of the schedule, not the architecture.

Table 2: Where the gain lives. Per-class hit@0.10 on the headline mix (five seeds, same basis as Table 1); hard routing’s unparsed share and its rates on parsed outputs; the fraction of click predictions that are the clamped type target (0, 0), and click hit@0.10 excluding them. Every model scores zero on the 35 *type* events because the raw target is off-screen.

Variant	click	scroll	unparsed click / scroll	click, parsed only	clicks at (0, 0)	click, excluding those
A: flat	0.256 ± 0.062	0.304 ± 0.034	0% / 0%	0.256	4.3% (13% on the worst seed)	0.266
B: aux. loss	0.357 ± 0.026	0.278 ± 0.018	0% / 0%	0.357	2.4%	0.366
C: hard routing	0.350 ± 0.061	0.148 ± 0.073	1.4% / 13.9%	0.356 (scroll 0.172)	0.0%	0.350
D-hook: additive	0.321 ± 0.064	0.357 ± 0.039	0% / 0%	0.321	0.0%	0.321
D-token: prepended	0.269 ± 0.069	0.304 ± 0.090	0% / 0%	0.269	0.0%	0.269
D-text: prompt word	0.348 ± 0.036	0.361 ± 0.036	0% / 0%	0.348	0.0%	0.348

5.2 How much of the gain is protection from a degenerate class?

Table 2 shows where the gain lives. It is a click gain: B and D-text gain nine to ten points on clicks, D-hook six, and C ten while paying sixteen on scrolls, because forcing the word *scroll* derails the gesture (it fails to parse 14% of scroll outputs against 1% of click outputs, and its parsed scroll rate is still 0.172, so most of that deficit is misplacement). D-hook and D-text improve both classes. The gain comes from our own preprocessing: the serializer clamps AITW’s (−1, −1) touch point for type events to (0, 0), so the mixed stream contains a class with a fixed target that the flat model must predict with no indication of the type, a situation no deployed action format creates (native formats emit no coordinate for a type action, or emit the type first). The simplest story, that the baseline emits the clamped target for clicks, accounts for little: it does so for 4.3% of clicks, B for 2.4%, and the models conditioned at inference never, and excluding those predictions moves the baseline’s click rate only from 0.256 to 0.266. The damage is broader: on the seed for which we logged predictions, the baseline’s clicks lie 0.10 of the screen width left of their targets on average (0.15 among misses) and 7% fall within 0.10 of the origin, against −0.02 and 0.6% for D-hook on the same screens, so the class pulls its clicks toward the clamped point without making it emit that point.

Retraining measures how much of the gain is protection from that class: we retrain all six variants on the same training slice with the type events removed and score the identical 250 validation examples (five seeds for A, B and D-hook, three for the rest; Table 7, appendix). Removing the class raises the flat baseline from 0.229 to 0.296 hit@0.10 (+0.067, [+0.044, +0.091], seed $p = 0.04$; on clicks from 0.256 to 0.355) and leaves every conditioned variant within 0.025 of where it was, with intervals through zero. On that stream no mechanism beats the baseline on the headline metric: B’s margin is +0.002 ([-0.014, +0.018], seed $p = 0.93$) and D-hook’s +0.001 ([-0.020, +0.022], seed $p = 0.97$), intervals tight enough to exclude a two-point effect, while C, D-token and D-text are at −0.028, −0.024 and −0.015. The gain on hit@0.10 is therefore accounted for by the class. What survives is smaller and about a different quantity: B shortens the average miss by −0.023 ([-0.030, −0.016], seed $p = 0.02$), the only clean-stream contrast that passes both of our tests, and gains +0.035 hit@0.25 ([+0.012, +0.059], seed $p = 0.16$) on the bootstrap alone. With the control in the next section, we read this as follows: on the mixed stream, protection from a class our serialization made degenerate accounts for the hit@0.10 gain, and we do not claim a spatial prior. Two things keep the question open. Type supervision still buys something on a clean stream, just not precise localization: the auxiliary loss reliably shortens the average miss without hitting the target more often, which is what a model with a better prior on plausible regions and no better fix on the exact widget would look like. And the result rests on one dataset, one serialization, and validation slices of a few hundred steps from a few dozen episodes, so we report it as a hazard we found in our own pipeline rather than a general property of type supervision.

5.3 Control: no gain on taps and swipes

On *taps_and_swipes* (Table 3), whose training stream has no degenerate class, B and D-hook are within noise of the baseline (−0.013 and −0.018 hit@0.10, intervals [−0.042, +0.013] and [−0.056, +0.019]), so the data admit harms of up to five points as well as no effect; D-text is worse by −0.047, with an interval that also includes zero ([-0.094, +0.002], seed $p = 0.09$). C is worse by −0.057 with an interval that excludes zero ([-0.101, −0.011]) but a seed-level p of 0.22, and a large increase in mean distance; on parsed outputs its deficit shrinks to −0.045 (0.337 against 0.382), so parse failures explain part of it and not all. The Multimodal-Mind2Web control is at floor (a tenth

Table 3: Control: AITW taps_and_swipes ($n_{\text{train}}=1000$, $n_{\text{val}}=200$, 3 seeds), a training stream without the degenerate class. Same statistics as Table 1.

Variant	hit@0.05	hit@0.10	hit@0.25	mean L2 ↓	Δ hit@0.10 vs. A [episode CI]	seed-level p
A: flat	0.243 $\pm$ 0.020	0.382 $\pm$ 0.015	0.720 $\pm$ 0.017	0.185 $\pm$ 0.003		
B: aux. loss	0.225 $\pm$ 0.015	0.368 $\pm$ 0.012	0.730 $\pm$ 0.039	0.173 $\pm$ 0.011	-0.013 [-0.042, +0.013]	0.456
C: hard routing	0.217 $\pm$ 0.016	0.325 $\pm$ 0.040	0.635 $\pm$ 0.064	0.271 $\pm$ 0.068	-0.057 [-0.101, -0.011]*	0.215
D-hook: additive	0.235 $\pm$ 0.022	0.363 $\pm$ 0.021	0.725 $\pm$ 0.043	0.182 $\pm$ 0.009	-0.018 [-0.056, +0.019]	0.334
D-text: word in prompt	0.210 $\pm$ 0.022	0.335 $\pm$ 0.031	0.665 $\pm$ 0.013	0.199 $\pm$ 0.003	-0.047 [-0.094, +0.002]	0.088

Table 4: Interventions: the same trained model decoded with the gold action embedding, a wrong one (cyclic; click→type), a wrong one swapping only clicks and scrolls, the table zeroed, and every row replaced by the class mean. Five seeds $\times$ 250 examples; Δ = gold – condition, 95% episode-cluster CI, seed-level paired p ; flat A on the same examples is the reference row.

Model	Conditioning	hit@0.10	click hit@0.10	scroll hit@0.10	Δ gold – cond. (hit@0.10)	seed p
D-hook: additive	gold type	0.275 $\pm$ 0.040	0.310 $\pm$ 0.060	0.357 $\pm$ 0.052		
	wrong (cyclic)	0.054 $\pm$ 0.009	0.006 $\pm$ 0.000	0.270 $\pm$ 0.050	+0.222 [+0.182, +0.263]***	0.000
	wrong (click↔scroll)	0.099 $\pm$ 0.056	0.073 $\pm$ 0.087	0.270 $\pm$ 0.050	+0.176 [+0.134, +0.221]***	0.003
	zeroed	0.248 $\pm$ 0.032	0.288 $\pm$ 0.053	0.291 $\pm$ 0.050	+0.027 [-0.006, +0.065]	0.224
	class mean	0.206 $\pm$ 0.058	0.217 $\pm$ 0.099	0.322 $\pm$ 0.064	+0.070 [+0.034, +0.109]***	0.133
	flat A, same examples	0.229 $\pm$ 0.043	0.256	0.304	+0.046 [+0.016, +0.081]**	0.032
D-token: prepended	gold type	0.242 $\pm$ 0.054	0.272 $\pm$ 0.079	0.313 $\pm$ 0.099		
	wrong (cyclic)	0.079 $\pm$ 0.012	0.036 $\pm$ 0.019	0.300 $\pm$ 0.054	+0.162 [+0.127, +0.197]***	0.001
	wrong (click↔scroll)	0.155 $\pm$ 0.043	0.148 $\pm$ 0.063	0.300 $\pm$ 0.054	+0.086 [+0.050, +0.125]***	0.073
	zeroed	0.162 $\pm$ 0.072	0.160 $\pm$ 0.112	0.291 $\pm$ 0.033	+0.080 [+0.054, +0.107]***	0.120
	class mean	0.169 $\pm$ 0.043	0.169 $\pm$ 0.067	0.296 $\pm$ 0.025	+0.073 [+0.046, +0.100]***	0.091
	flat A, same examples	0.229 $\pm$ 0.043	0.256	0.304	+0.013 [-0.013, +0.039]	0.374

of predicted points inside the gold element for both A and D-hook) and is reported in the appendix only.

5.4 End to end with predicted types: the deployment number

To close the loop we train D-hook, fit the Stage-1 MLP on frozen features of its training examples (200 full-batch epochs, no checkpoint selection), and evaluate it on the same validation examples with gold and with predicted types (three seeds). Stage 1 reaches 0.843 accuracy against a majority class of 0.676. The oracle pipeline scores 0.292 hit@0.10, the predicted-type pipeline 0.271, the flat baseline 0.255. The predicted pipeline’s margin over the baseline is not established (+0.016, [-0.017, +0.052], seed $p = 0.44$); the oracle-to-predicted gap is (+0.021, [+0.003, +0.041], seed $p = 0.07$). Classifier error removes more than half of the oracle gain, and Section 6.1 shows why it is the risk: a wrong type costs far more than a missing one.

6 Mechanism

6.1 Interventions: which signals are read, and what remains when they are removed

D-token might ignore its slot, and D-hook’s gain might be a training effect that leaves a vector the model never reads. To test both on equal terms we retrain each at the headline setting (shared learning rate, five seeds) and decode every trained model under five conditionings: the gold type; a wrong type from a cyclic permutation of the classes present (click to type, type to scroll, scroll to click); a second wrong map swapping only clicks and scrolls, so no groundable example is routed to the degenerate class; the table zeroed; and every row replaced by the mean of the rows present. Table 4 reports the results with the flat baseline on the same examples (an earlier three-condition test, Table 10, agrees). D-hook’s learned rows are small (norm 0.04 against a mean token-embedding norm of 0.58; D-token’s have norm 0.78), and the model reads them. A wrong type collapses D-hook from 0.275 to 0.054 under the cyclic map and to 0.099 under the click-scroll map (gold minus wrong: +0.222, [+0.182, +0.263], and +0.176, [+0.134, +0.221]; seed $p < 0.001$ and 0.003), so the collapse is not an artifact of routing clicks to the degenerate class. The class-mean vector costs seven points (+0.070, [+0.034, +0.109]), though the seed-level test does not reach significance ($p = 0.13$). Zeroing the table costs +0.027 with an interval from -0.006 to +0.065 ($p = 0.22$): the data are consistent with no cost and with a cost of six points, most of the +0.046 ([+0.016, +0.081])

that gold conditioning gives over the baseline on these examples. What can be stated is a bound: the zeroed additive model is no more than one point below the flat baseline (zeroed minus A: $+0.019$, $[-0.006, +0.044]$). A single-seed attention run that we had first read as showing the embedding could be removed without loss (Appendix H) was underpowered.

D-token differs. Every perturbation lowers its accuracy with a bootstrap interval excluding zero, including the class mean ($+0.073$, $[+0.046, +0.100]$), though with five seeds only the cyclic wrong type also passes the seed-level test ($p = 0.001$). Its zeroed model is below the flat baseline by -0.067 ($[-0.089, -0.044]$, seed $p < 0.01$), whereas the zeroed D-hook is not: adaptation with a prepended token makes the model depend on it, adaptation with an additive vector does not, even though the vector is used. The prompt word behaves the same way (Table 11; three seeds): a wrong word collapses D-text from 0.312 to 0.076 and 0.085 (gold minus wrong $+0.236$ and $+0.227$, seed $p = 0.017$ and 0.014), and with no word it scores 0.205, below the baseline by -0.049 ($[-0.079, -0.018]$). Under a wrong type, the failure a deployed classifier produces, the additive model is the most brittle: 0.054 and 0.099 against D-token’s 0.079 and 0.155 and D-text’s 0.076 and 0.085. Conditioning acts almost entirely on clicks in every mechanism, and attention at the y-predicting token moves with accuracy under these interventions on one seed (Appendix H).

6.2 A positional-encoding failure that inverted a conclusion

Our first implementation of the prepended token replaced the slot’s embedding by building `inputs_embeds` and passing that instead of `input_ids`. On `taps_and_swipes` it scored 0.298 ± 0.026 hit@0.10 against the flat baseline’s 0.390 ± 0.010 in the same runs, and the same slot with a frozen random embedding scored 0.290, which pointed at the injection path. Qwen2-VL derives its three-dimensional rotary positions from `input_ids`; given `inputs_embeds` alone, transformers falls back to one-dimensional positions replicated across the three axes, and the image tokens lose their two-dimensional layout (Appendix G; Hugging Face Transformers contributors, 2024). A forward hook with `input_ids` intact removed the deficit. We inferred the cause from the library source and the frozen-embedding control rather than from position ids in the failing runs, and record it because the failure is silent.

7 Limitations

The study is small: one 2B backbone, LoRA adapters, validation slices of 200 to 250 steps from 35 to 50 episodes, one learning rate and adapter configuration for every variant, a stored-order split rather than the official episode split, and no action history. Per-seed variance is large, so the conclusions rest on paired statistics at two levels, neither clean: the episode bootstrap ignores that seeds share a data order, and the seed-level test has five or three units. The prepended token’s failure at the shared learning rate is a schedule effect and the faster-trained token has three seeds; we did not train a frozen-slot control on the fixed path, and the class-mean intervention is matched to D-token in direction but not in norm. The clean-stream result bounds rather than excludes a small advantage, and how far the degenerate-class hazard generalizes is open: whether other corpora encode non-spatial actions the same damaging way, whether the effect scales, and whether the residual mean-distance advantage holds on larger slices. Those are the experiments we would run next. The end-to-end result is descriptive, the scaling study is uncorrected on moving slices, the attention analysis uses one seed and one layer, and Mind2Web is at floor. Code, split indices, prompts, and the per-example logs behind every table are available at <https://github.com/aadcha/action-conditioned-gui-agent>.

8 Conclusion

An auxiliary loss, an additive embedding, and a word in the prompt each help a small grounding model on a stream our serialization gave a degenerate class; none is established to help once that class is removed or on taps and swipes, and the prepended token we set out to validate helps only when its table is trained faster than the shared schedule, and then no better than the simpler mechanisms. Both learned embeddings are read at inference: zeroed, the additive model stays within a point of the baseline and the prepended one falls seven points below, and a wrong type collapses both, so the classifier is the risk; with an 84% one the pipeline’s margin over a flat model is not

established. The practical lessons: audit what a serialization does to every class before supervising a coordinate on it, prefer the prompt word or the auxiliary loss over new parameters, perturb the conditioning to test whether it is read, and keep `input_ids` on the path whenever conditioning touches the embeddings.

References

- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.
- Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Cărbune, Jason Lin, Jindong Chen, and Abhanshu Sharma. ScreenAI: A vision-language model for UI and infographics understanding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. DigiRL: Training in-the-wild device-control agents with autonomous reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997. doi: 10.1023/A:1007379606734.
- Keqin Chen, Zhao Zhang, Weili Zeng, Richong Zhang, Feng Zhu, and Rui Zhao. Shikra: Unleashing multimodal LLM’s referential dialogue magic. arXiv preprint arXiv:2306.15195, 2023.
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, 2024. doi: 10.18653/v1/2024.acl-long.505.
- Filippos Christianos, Georgios Papoudakis, Thomas Coste, Jianye Hao, Jun Wang, and Kun Shao. Lightweight neural app control. In *International Conference on Learning Representations (ICLR)*, 2025.
- Christopher Clark, Yue Yang, Jae Sung Park, Zixian Ma, Jieyu Zhang, Rohun Tripathi, Mohammadreza Salehi, Sangho Lee, Taira Anderson, et al. MolmoPoint: Better pointing for VLMs with grounding tokens. arXiv preprint arXiv:2603.28069, 2026.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Number 57 in Monographs on Statistics and Applied Probability. Chapman & Hall, New York, 1993.
- Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoub, Dinesh Manocha, and Tianyi Zhou. HallusionBench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14375–14385, 2024. doi: 10.1109/CVPR52733.2024.01363.
- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. CogAgent: A visual language model for GUI agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14281–14290, 2024. doi: 10.1109/CVPR52733.2024.01354.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Hugging Face Transformers contributors. Qwen2-VL used to work with `inputs_embeds` instead of `input_ids`, but no more. GitHub issue #35463 (opened December 31, 2024) and pull request #35466, <https://github.com/huggingface/transformers/issues/35463>, 2024.
- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. ScreenSpot-Pro: GUI grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM)*, pages 8778–8786, 2025. doi: 10.1145/3746027.3755688.
- Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on UI control agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 292–305, 2023. doi: 10.18653/v1/2023.emnlp-main.20.
- Jiapeng Lin, Fei Shen, Junzhe Li, Ping Nie, Fei Yu, Ming Li, and Haizhou Li. What happens before decoding? prefill determines GUI grounding in VLMs. arXiv preprint arXiv:2605.12549, 2026.
- Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. ShowUI: One vision-language-action model for GUI visual agent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19498–19508, 2025. doi: 10.1109/CVPR52734.2025.01816.
- Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Long, Jiadai Sun, Jiaqi Wang, et al. AutoGLM: Autonomous foundation agents for GUIs. arXiv preprint arXiv:2411.00820, 2024.
- Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for pure vision based GUI agent. arXiv preprint arXiv:2408.00203, 2024.
- Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, Guanjing Xiong, and Hongsheng Li. UI-R1: Enhancing efficient action prediction of GUI agents by reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 17608–17616, 2026. doi: 10.1609/aaai.v40i21.38816.
- Xinbei Ma, Zhuosheng Zhang, and Hai Zhao. CoCo-Agent: A comprehensive cognitive MLLM agent for smartphone GUI automation. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

- Pawel Pawlowski, Krystian Zawistowski, Wojciech Lapacz, Adam Wiacek, Marcin Skorupa, Sebastien Postansque, and Jakub Hoscilowicz. TinyClick: Single-turn agent for empowering GUI automation. arXiv preprint arXiv:2410.11871, 2024.
- Zhiliang Peng, Wenhui Wang, Li Dong, Yaru Hao, Shaohan Huang, Shuming Ma, and Furu Wei. Kosmos-2: Grounding multimodal large language models to the world. In *International Conference on Learning Representations (ICLR)*, 2024.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjuan Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering automated GUI interaction with native agents. arXiv preprint arXiv:2501.12326, 2025.
- Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for Android device control. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.
- Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution. arXiv preprint arXiv:2409.12191, 2024.
- Suyuchen Wang, Tianyu Zhang, Ahmed Masry, Christopher Pal, Spandana Gella, Bang Liu, and Perouz Taslakian. Improving GUI grounding with explicit position-to-coordinate mapping. arXiv preprint arXiv:2510.03230, 2025a.
- Yiqin Wang, Haoji Zhang, Jingqi Tian, and Yansong Tang. Ponder & Press: Advancing visual GUI agent towards general computer control. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 1461–1473, 2025b. doi: 10.18653/v1/2025.findings-acl.76.
- Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, Si Qin, Lars Liden, Qingwei Lin, Huan Zhang, Tong Zhang, Jianbing Zhang, Dongmei Zhang, and Jianfeng Gao. GUI-Actor: Coordinate-free visual grounding for GUI agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.
- Qingyuan Wu, Jianheng Liu, Jianye Hao, Jun Wang, and Kun Shao. Advancing autonomous VLM agents via variational subgoal-conditioned reinforcement learning. arXiv preprint arXiv:2502.07949, 2025b. Presented at the NeurIPS 2025 Workshop on Vision Language Models: Challenges of Real World Deployment.
- Zhe Wu, Hongjin Lu, Junliang Xing, Changhao Zhang, Yuxuan Li, Yin Zhu, Yuhao Yang, Yuheng Jing, Kai Li, Kun Shao, Jianye Hao, et al. Hi-Agent: Hierarchical vision-language agents for mobile device control. arXiv preprint arXiv:2510.14388, 2025c.

- Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A foundation action model for generalist GUI agents. In *International Conference on Learning Representations (ICLR)*, 2025d.
- Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguis: Unified pure vision agents for autonomous GUI interaction. In *International Conference on Machine Learning (ICML)*, 2025.
- Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. arXiv preprint arXiv:2310.11441, 2023.
- Rui Yang, Qianhui Wu, Zhaoyang Wang, Hanyang Chen, Ke Yang, Hao Cheng, Huaxiu Yao, Baolin Peng, Huan Zhang, Jianfeng Gao, and Tong Zhang. GUI-Libra: Training native GUI agents to reason and act with action-aware supervision and partially verifiable RL. arXiv preprint arXiv:2602.22190, 2026.
- Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-UI: Visual grounding for GUI instructions. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22418–22433, 2025a. doi: 10.18653/v1/2025.findings-acl.1152.
- Zhen Yang, Zi-Yi Dou, Di Feng, Forrest Huang, Anh Nguyen, Keen You, Omar Attia, Yuhao Yang, Michael Feng, Haotian Zhang, et al. Ferret-UI Lite: Lessons from building small on-device GUI agents. arXiv preprint arXiv:2509.26539, 2025b.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Haoxuan You, Haotian Zhang, Zhe Gan, Xianzhi Du, Bowen Zhang, Zirui Wang, Liangliang Cao, Shih-Fu Chang, and Yinfei Yang. Ferret: Refer and ground anything anywhere at any granularity. In *International Conference on Learning Representations (ICLR)*, 2024a.
- Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-UI: Grounded mobile UI understanding with multimodal LLMs. In *European Conference on Computer Vision (ECCV)*, pages 240–255, 2024b. doi: 10.1007/978-3-031-73039-9_14.
- Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. AppAgent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI)*, pages 1–20, 2025. doi: 10.1145/3706598.3713600.
- Zhuosheng Zhang and Aston Zhang. You only look at screens: Multimodal chain-of-action agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3132–3149, 2024. doi: 10.18653/v1/2024.findings-acl.186.
- Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *International Conference on Machine Learning (ICML)*, 2024.
- Shijie Zhou, Viet Dac Lai, Hao Tan, Jihyung Kil, Wanrong Zhu, Changyou Chen, and Ruiyi Zhang. GUI-AIMA: Aligning intrinsic multimodal attention with a context anchor for GUI grounding. arXiv preprint arXiv:2511.00810, 2025.

A Training and evaluation details

Data. AITW screenshots in the `cjfcst/AITW_General` mirror are stored as raw RGB byte buffers without an image header; we decode them by matching the buffer length against seven observed resolutions (for example 540×1140 and 412×732) with a portrait-aspect fallback. Mind2Web screenshots are downsampled so that no image exceeds one megapixel. Examples are drawn from the AITW training stream in stored order, filtered to the mix’s action labels; the first n eligible steps form the training slice and the next 250 (200 for the control mix) form validation. Consecutive steps belong to the same episodes, so a validation slice spans a few dozen episodes (35 to 50 across the sizes we use) and shares its boundary episode with training (two to ten steps, four at the headline size). Table 8 lists the class mix of each validation slice.

Prompts. Every Stage-2 prompt is the chat-template user turn containing the screenshot, the line `Goal: {goal}`, and the instruction `Predict the action coordinate.`; the assistant turn is the answer `(x, y)` with integers on the 0–999 grid. C uses `Predict the action type and coordinate.` and is trained to answer with the action word before the coordinate, `click (x, y)`; D-text prefixes the user turn with `Action: {word}.`; D-token places `<|action_slot|>` at the start of the text, and exactly one slot per sequence is asserted at every step.

Optimization. AdamW with learning rate 2×10^{-5} and weight decay 0.01, batch size 1, gradient clipping at 1.0, two epochs, a fixed per-epoch shuffle seeded by the run seed. The action-embedding table (D-hook, D-token) and the auxiliary head (B) share the base learning rate unless stated otherwise. Evaluation is greedy with at most 16 new tokens; the coordinate is parsed with a regular expression and an unparseable output counts as a miss at distance $\sqrt{2}$. Greedy evaluation is deterministic for a trained model; models in different tables that share a seed are separately trained copies (Table 10 and the intervention runs retrain the variant), and their numbers differ by training nondeterminism.

Compute. Everything runs on single NVIDIA L4 GPUs. A Stage-2 run at 1,200 training examples takes about 35 minutes; the full study, including retracted and re-run cells, cost about 150 GPU-hours.

Statistics. For each contrast we align the per-example distances of the two variants on every shared seed. The episode bootstrap resamples the episodes of the validation slice with replacement, keeps every step and every seed of a sampled episode together, recomputes the mean difference 10,000 times, and reports the percentile interval and the two-sided bootstrap p . The seed-level test is a paired t -test on the per-seed means. Clustering on examples instead of episodes would add one star in the paper (hard routing’s headline gain, example-cluster interval $[+0.001, +0.069]$); the (seed, example)-unit bootstrap and its permutation counterpart give intervals about a third narrower still and are reported in the released logs. Every star in this paper was recomputed on the episode basis.

B Stage 1, Mind2Web, and per-class tables

Table 5: Stage 1 action-type macro-F1 (three seeds). On Mind2Web the instruction names the action, so the screenshot adds little; on AITW the instruction describes a goal and the screenshot is what separates the classes.

Features	Mind2Web (2 classes)	AITW (5 classes)
text only	0.597 ± 0.011	0.141 ± 0.032
text + screenshot	0.605 ± 0.016	0.555 ± 0.036
zeroed (sanity)	0.469 ± 0.000	0.110 ± 0.051
TF-IDF + logistic regression	0.622	0.168
majority class	0.472	0.140

Table 6: Multimodal-Mind2Web grounding control (1,000 training and 250 evaluation steps, three seeds, seed-level only). The target is the center of the gold element; hit@bbox is the fraction of predicted points inside the element. Both models are at floor.

Variant	hit@0.10	hit@0.25	hit@bbox	mean L2 ↓
A: flat	0.365 ± 0.031	0.645 ± 0.035	0.095 ± 0.018	0.234 ± 0.023
D-hook: additive	0.355 ± 0.032	0.664 ± 0.052	0.104 ± 0.034	0.233 ± 0.027

C Type events removed from training

Table 7 retrains all six variants on the headline training slice with the 35 type events removed, scored on the identical 250 validation examples, so the with-type and type-removed rows of each variant are paired example by example.

Table 7: Removing *type* events from the training slice only, scored on the identical headline validation slice (3 to 5 seeds; each contrast uses the seeds its two conditions share, given per row). Deltas are paired episode-cluster bootstraps (every step and seed of a sampled episode kept together); seed-level p in parentheses.

Variant	Training stream	seeds	hit@0.10	click hit@0.10	scroll hit@0.10	Δ hit@0.10 vs. A (same stream)
A: flat	with type	5	0.229 ± 0.043	0.256 ± 0.062	0.304 ± 0.034	
A: flat	type removed	5	0.296 ± 0.019	0.355 ± 0.031	0.304 ± 0.051	
B: aux. loss	with type	5	0.293 ± 0.021	0.357 ± 0.026	0.278 ± 0.018	
B: aux. loss	type removed	5	0.298 ± 0.038	0.368 ± 0.069	0.265 ± 0.058	+0.002 [-0.014, +0.018] (0.926)
C: hard routing	with type	5	0.264 ± 0.045	0.350 ± 0.061	0.148 ± 0.073	
C: hard routing	type removed	3	0.269 ± 0.033	0.355 ± 0.039	0.159 ± 0.126	-0.028 [-0.071, +0.013] (0.118)
D-hook: additive	with type	5	0.282 ± 0.041	0.321 ± 0.064	0.357 ± 0.039	
D-hook: additive	type removed	5	0.297 ± 0.039	0.341 ± 0.061	0.361 ± 0.029	+0.001 [-0.020, +0.022] (0.973)
D-token: prepended	with type	5	0.238 ± 0.049	0.269 ± 0.069	0.304 ± 0.090	
D-token: prepended	type removed	3	0.273 ± 0.015	0.333 ± 0.028	0.261 ± 0.038	-0.024 [-0.040, -0.008]** (0.402)
D-text: word in prompt	with type	5	0.302 ± 0.028	0.348 ± 0.036	0.361 ± 0.036	
D-text: word in prompt	type removed	3	0.283 ± 0.017	0.318 ± 0.015	0.370 ± 0.038	-0.015 [-0.064, +0.036] (0.490)

D Scaling study

Figure 2 and Table 8 extend the comparison of A, B, and D-hook to $n \in \{300, 500, 800, 1200, 2500, 5000\}$ with three seeds per size. Each size is scored on a different validation slice whose scroll count ranges from 12 to 71, and B hurts scrolls while D-hook helps them (Table 2), so only within-size contrasts are read, and the 24 contrasts are uncorrected: D-hook – A excludes zero at five of the six sizes (all but 500) on hit@0.10, B – A at 300 and 1,200 and is negative at 800, and with three seeds the seed-level tests pass only at 300 and 5,000 for D-hook and at 300 and 1,200 for B. An earlier single-seed curve had suggested that every conditioning gain shrinks with data, which three seeds do not support.

Table 8: Conditioning advantage vs. training-set size (AITW all_with_coords, 3 seeds per cell). Intervals are 95% episode-cluster bootstraps (every step and seed of a sampled episode kept together); stars from the same bootstrap. Absolute values are not comparable across rows because the validation slice moves with n ; its click/scroll/type counts are given per row. The study is descriptive: 24 contrasts, no multiplicity correction.

n_{train}	val mix (c/s/t)	A hit@0.10	B – A	D-hook – A	B – A (hit@0.25)	D-hook – A (hit@0.25)
300	170/33/47	0.139 ± 0.032	+0.067 [+0.029, +0.102]**	+0.069 [+0.040, +0.098]***	-0.055 [-0.101, -0.011]*	+0.067 [+0.031, +0.102]***
500	198/12/40	0.315 ± 0.059	+0.011 [-0.020, +0.040]	+0.001 [-0.032, +0.033]	+0.004 [-0.028, +0.034]	+0.059 [+0.030, +0.085]***
800	141/71/38	0.261 ± 0.020	-0.040 [-0.077, -0.006]*	+0.048 [+0.010, +0.085]*	-0.039 [-0.081, +0.001]	+0.064 [+0.009, +0.111]*
1200	169/46/35	0.255 ± 0.026	+0.051 [+0.020, +0.081]***	+0.045 [+0.011, +0.085]***	+0.071 [+0.037, +0.103]***	+0.055 [+0.019, +0.090]**
2500	166/53/31	0.244 ± 0.037	-0.012 [-0.037, +0.010]	+0.037 [+0.015, +0.061]**	-0.008 [-0.046, +0.026]	+0.104 [+0.060, +0.144]***
5000	187/29/34	0.363 ± 0.009	+0.005 [-0.026, +0.036]	+0.043 [+0.006, +0.079]*	-0.023 [-0.055, +0.008]	+0.032 [-0.005, +0.069]

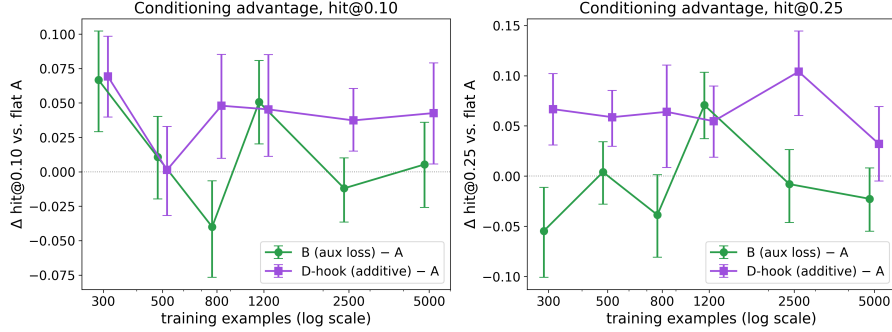


Figure 2: Conditioning advantage over the flat baseline as a function of training-set size on all_with_coords, three seeds per cell, 95% episode-clustered bootstrap intervals. Each size is scored on a different validation slice with a different class mix, so only within-size differences are meaningful, and 24 contrasts are shown without correction.

E D-token: learning-rate sweep and the earlier three-condition test

Table 9: D-token with the action-embedding table trained at $1\times$, $10\times$, and $100\times$ the base learning rate on the headline slice. Displacement is the mean distance the rows of the classes present (click, type, scroll) moved from their $\mathcal{N}(0, 0.02^2)$ initialization, whose norm is 0.78; the $1\times$ value comes from the type-removed runs, which share the code path and learning rate. Δ vs. A uses the shared seeds with an episode-cluster interval.

Table LR	seeds	hit@0.10	hit@0.25	click hit@0.10	Δ hit@0.10 vs. A	seed p	row displacement
shared (2×10^{-5})	5	0.238 ± 0.049	0.504 ± 0.055	0.269 ± 0.069	$+0.009 [-0.019, +0.036]$	0.465	0.016
2×10^{-4} ($10\times$)	3	0.295 ± 0.041	0.561 ± 0.045	0.329 ± 0.048	$+0.040 [+0.005, +0.076]^*$	0.157	0.276
2×10^{-3} ($100\times$)	3	0.283 ± 0.032	0.544 ± 0.022	0.318 ± 0.030	$+0.028 [-0.008, +0.065]$	0.020	1.874

Table 9 is the sweep behind Section 5.1: the same D-token architecture with only the action-embedding table’s learning rate changed, three seeds per rate against the shared seeds of the flat baseline. The displacement column is measured on runs that log the table before and after training; the headline runs predate that logging, so the shared-rate value comes from the type-removed runs, which use the same rate and code path. Table 10 is the test that preceded Table 4: separately trained

Table 10: D-token causal-use test: the same trained model evaluated with the gold action embedding, a wrong (cyclically permuted) one, and a zeroed one (5 seeds; paired episode-cluster bootstrap intervals).

Conditioning	hit@0.05	hit@0.10	hit@0.25	mean L2 $\downarrow$
gold	0.140 ± 0.059	0.236 ± 0.058	0.502 ± 0.045	0.386 ± 0.020
wrong	0.014 ± 0.004	0.079 ± 0.013	0.200 ± 0.027	0.728 ± 0.081
zero	0.074 ± 0.055	0.167 ± 0.066	0.416 ± 0.103	0.489 ± 0.059
gold – wrong		$+0.157 [+0.120, +0.192]^{***}$	$+0.302 [+0.258, +0.342]^{***}$	$-0.342 [-0.366, -0.314]^{***}$
gold – zero		$+0.069 [+0.042, +0.096]^{***}$	$+0.086 [+0.062, +0.109]^{***}$	$-0.103 [-0.115, -0.091]^{***}$

D-token models (five seeds), decoded with the gold embedding, the cyclic wrong map, and the table zeroed. Its numbers differ from the D-token rows of Table 4 by training nondeterminism and agree in every conclusion. The class-mean control of Table 4 is matched to D-token in direction but not in norm: the mean of three near-orthogonal rows of norm 0.78 has norm about 0.45, so a fresh $\mathcal{N}(0, 0.02^2)$ draw would be the matched control, and we did not run it.

F Interventions on the prompt word

Table 11 applies the protocol of Section 6.1 to D-text, whose conditioning is a word in the prompt rather than a learned vector: the same trained model is decoded with the gold word, a wrong word under the two maps, and no word at all.

Table 11: Interventions on D-text: the same trained model decoded with the gold action word, a wrong word (cyclic; click→type), a wrong word swapping only clicks and scrolls, and no action word (3 seeds × 250 examples; Δ = gold – condition, 95% episode-cluster CI, seed-level paired p ; flat A on the same examples is the reference row).

Conditioning	hit@0.10	click hit@0.10	scroll hit@0.10	Δ gold – cond. (hit@0.10)	seed p
gold word	0.312 $\pm$ 0.045	0.357 $\pm$ 0.056	0.384 $\pm$ 0.045		
wrong word (cyclic)	0.076 $\pm$ 0.011	0.028 $\pm$ 0.003	0.312 $\pm$ 0.050	+0.236 [+0.183, +0.289]***	0.017
wrong word (click↔scroll)	0.085 $\pm$ 0.006	0.041 $\pm$ 0.006	0.312 $\pm$ 0.050	+0.227 [+0.174, +0.281]***	0.014
no action word	0.205 $\pm$ 0.067	0.217 $\pm$ 0.102	0.319 $\pm$ 0.055	+0.107 [+0.063, +0.152]***	0.241
flat A, same examples	0.229 $\pm$ 0.043	0.256	0.304	+0.057 [+0.018, +0.099]**	0.083

G The inputs_embeds failure in detail

Qwen2-VL locates the image tokens in `input_ids` to assign each visual token a temporal, height, and width position for its three-dimensional rotary embedding. When a caller supplies `inputs_embeds` and no `input_ids`, the model cannot perform that lookup; in the transformers releases our runs used (pinned at 4.45 or later; the current 5.9 release behaves the same) the positions become a one-dimensional sequence index replicated across the three axes, so every image token is positioned as text and the two-dimensional layout the grounding task depends on is lost. A caller on that path must also merge the visual features into the embeddings itself, a second opportunity for silent error. We inferred the cause from the code and from the frozen-embedding control described in Section 6.2 rather than by dumping position ids at the time. The same dependence on `input_ids` was reported as a generation crash in December 2024 [Hugging Face Transformers contributors, 2024]; the silent degradation when images are present is the case a fine-tuning practitioner meets. In the set of runs where the failure appeared, the fixed additive variant scored 0.392 ± 0.043 hit@0.10 on `taps_and_swipes` against the flat baseline’s 0.390 ± 0.010 ; the re-run of that control with per-example logging (Table 3) gives 0.363 ± 0.021 against 0.382 ± 0.015 .

H Where the model looks

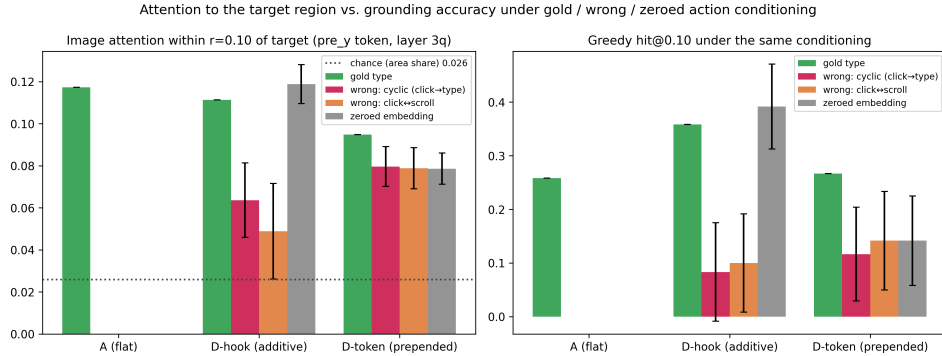


Figure 3: Same trained model, four conditionings, 120 validation screens, one seed per variant. Left: share of image attention within 0.10 of the target from the token that predicts the y coordinate, with the gold answer teacher-forced (3/4-depth layer, head-averaged); the dotted line is the share of image tokens within that radius. Right: greedy hit@0.10 under the same conditioning. Error bars are 95% paired-bootstrap intervals of the difference from gold over the 120 screens.

We measured where the models look, with the caveats that the measure is head-averaged attention at one layer, that attention is not information flow, and that teacher-forcing the gold answer places the gold x coordinate in the prefix of the token that predicts y. Under gold conditioning both embedding models put 10 to 11% of that token’s image attention within 0.10 of the target, against a chance share of 2.6%; the last prompt token, which our earlier visualizations used, puts about 4% there under every conditioning, so most of the localization we can see happens after the x coordinate is in the prefix. Teacher-forcing the model’s own answer instead of the gold one, so that the y-predicting

token sees the predicted x , lowers the target share only slightly (D-hook 0.092, D-token 0.085), so the localization is not an artifact of the supplied x . The flat baseline trained on the same seed localizes just as much (0.117 at the y -predicting token, 0.086 free-running) while grounding far less accurately on the same screens (0.258 vs. 0.358 hit@0.10), so conditioning does not add localization; the attention share is a property of the fine-tuned model, not of the type signal. Lin et al. [2026] report that grounding is largely decided during prefill; our probe positions are answer tokens under teacher forcing, so the two measurements are not directly comparable. The interventions move attention the same way they move accuracy on this seed (Figure 3, Table 12): a wrong type roughly halves the target share for D-hook, and zeroing costs D-token both attention and accuracy. On this one seed zeroing left D-hook’s attention and accuracy unchanged; we had first read that as showing the embedding could be removed without loss, and the five-seed study in Section 6.1 bounds the cost instead. Table 12 gives the target-attention share for the last prompt token (the one that predicts the opening parenthesis), the token that predicts the first x digit, and the token that predicts the first y digit.

Table 12: Share of image attention within 0.10 of the target (chance 0.026) at three probe positions, seed 42, 120 screens, 3/4-depth layer, gold answer teacher-forced.

Model	Position	gold	wrong (cyclic)	wrong (click↔scroll)	zero
A (flat)	last prompt token	0.038			
A (flat)	pre- y token	0.117			
D-hook	last prompt token	0.040	0.036	0.035	0.036
D-hook	pre- x token	0.039	0.036	0.037	0.039
D-hook	pre- y token	0.111	0.064	0.049	0.119
D-token	last prompt token	0.038	0.034	0.034	0.035
D-token	pre- x token	0.038	0.034	0.033	0.035
D-token	pre- y token	0.095	0.080	0.079	0.079

I Qualitative examples

Figure 4 shows six held-out screens with the predictions of the flat baseline and of D-hook.

AITW_all_with_coords, n_train=1200, seed 42: click hit@0.10 A=0.34 vs D-hook=0.39; of 169 clicks, 21 rescued (A miss, D hit), 13 hurt (A hit, D miss), 45 both correct, 90 both missed at $r=0.10$ (distances are normalized L2)

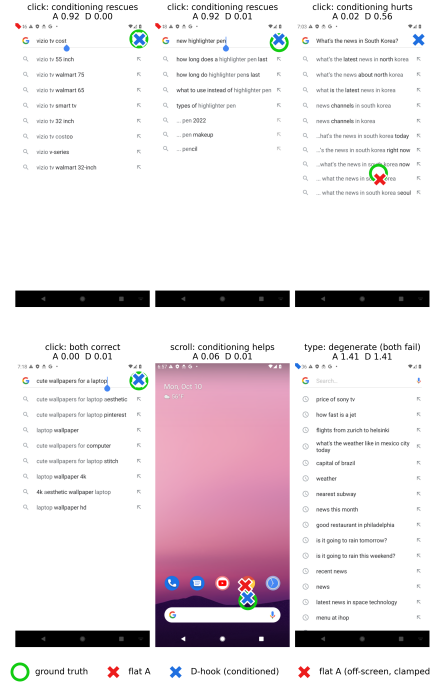


Figure 4: Predictions of the flat baseline (red) and D-hook (blue) against the target (green) on held-out AITW screens at the headline setting, one seed. The panels are a fixed spread of outcomes rather than a selection of wins: two rescues, a case where conditioning hurts, a case where both are right, a scroll, and a degenerate *type* example on which both models miss at distance $\sqrt{2}$. The title reports the base rates on this seed at $r = 0.10$: of 169 clicks, 21 are rescued by conditioning, 13 are hurt, 45 are correct under both models, and 90 are missed by both. In the rescue panels the baseline’s prediction is off-screen and drawn clamped at the corner.